

Design a model

Describe a database table as a Python class, and let Django build it for you.

The big idea

A backend must **remember** things between requests, so it keeps them in a **database**. A database table looks like a DataFrame: named **columns**, one **row** per record.

The difference: a table is **strict**. You decide up front each column's name, type and rules. In Django you write that plan as a Python class, a **model**, and Django builds the table.

What the tests do with your class

There's no function to call: the tests use your `Book` class directly.

```
Book._meta.get_field("author").max_length      # expects 100

book = Book.objects.create(title="Dune", author="Frank Herbert")
book.is_available      # expects True (nobody set it)
book.created_at        # expects a date and time, not None
str(book)              # expects "Dune by Frank Herbert"

# saved as Neuromancer, Dune, Foundation; expected back in this order:
# ["Dune", "Foundation", "Neuromancer"]

Book(title="1984", author="Orwell", published_year=1949).is_classic() # True
Book(title="X", author="Y", published_year=1950).is_classic()         # False
Book(title="Unknown", author="Anon").is_classic()                    # False
```

New words

model

A Python class that describes one table.

field

One attribute of a model. It becomes one column.

object

One row, as a Python value: `Book(title="Dune")`.

NULL / None

"No value": the database says `NULL`, Python says `None`.

method / self

A function inside a class; `self` is the one object it's called on.

migration

A file that updates the database tables to match your models.

Tools: field types

Each field is one line inside the class. The options set the rules for that column.

```
class Movie(models.Model):  
    name = models.CharField(max_length=150)           # text  
    runtime = models.PositiveIntegerField(null=True, blank=True) # optional  
    is_showing = models.BooleanField(default=False)    # yes / no  
    added_at = models.DateTimeField(auto_now_add=True) # automatic
```

null=True vs blank=True

null=True

- About the **database**
- The column may store `NULL`
- Python sees `None`
- Without it, saving no value fails
- Test 2 checks `field.null`

blank=True

- About **checking input**
- Forms and serializers allow it empty
- The user may leave it out
- Without it, input checks say "required"
- Test 2 checks `field.blank`

Tool: making and saving objects

`Book(...)` lives in memory only. `.save()` writes it; `objects.create(...)` does both.

```
book = Book(title="Emma")
print(book.title)           # Emma
print(book.pk)              # None (not saved yet)
book.save()
print(book.pk)              # 1 (the database gave it an id)
Book.objects.create(title="Beloved")
print(Book.objects.count()) # 2
```

Tool: class Meta and ordering

Settings for the whole table go in a small class inside the model.

```
class Movie(models.Model):
    name = models.CharField(max_length=150)

    class Meta:
        ordering = ["-name"]      # Z to A; ["name"] would be A to Z

Movie.objects.order_by("name")  # or sort just one query by hand
```

Tool: methods and `__str__`

A plain Python class (not a model) with both kinds of method:

```
class Film:
    name = "Alien"
    year = 1979

    def __str__(self):
        return f"{self.name} ({self.year})"

    def is_old(self):
        return self.year < 1990

film = Film()
print(film)           # Alien (1979)
print(film.is_old())  # True
```

Pattern: check for None first

`None < 1950` crashes with a `TypeError`. and stops early, so check for `None` first.

```
score = None
print(score is not None and score > 50)

score = 72
print(score is not None and score > 50)
```

OUTPUT

False

True

Your task, step by step

- 1 Add `author`: a `CharField` with `max_length=100` → **test 1**
- 2 Add `published_year`: a `PositiveIntegerField`, `null` and `blank` → **test 2**
- 3 Add `is_available`: a `BooleanField` with `default=True` → **test 3**
- 4 Add `created_at`: a `DateTimeField` with `auto_now_add=True` → **test 4**
- 5 `__str__` returns `title`, `by`, `author`, from `self` → **test 5**
- 6 Add `class Meta`: with `ordering = ["title"]` → **test 6**
- 7 `is_classic(self)`: a year that isn't `None` and is `< 1950` → **test 7**

Common mistakes

- A `CharField` without `max_length`: Django refuses to load the model
- Only `null=True` or only `blank=True`: test 2 needs both
- `ordering = "title"` (text) instead of a list, or outside `class Meta:`
- `published_year` instead of `self.published_year` inside a method
- `__str__` that prints instead of returning, or `<= 1950` instead of `< 1950`

Where you'll use this

A library app needs to store books, members and loans. Each one is a model, and every screen and API endpoint reads from them.

- After changing a model, run `python manage.py makemigrations`, then `migrate`
- The admin, forms and serializers all read their rules from your fields
- A good `__str__` turns `Book object (1)` into `Dune by Frank Herbert` in the admin

Check yourself

1. What is the difference between `null=True` and `blank=True`?
2. What does `auto_now_add=True` do, and when?
3. Where does `ordering` go, and why does it need square brackets?
4. Why check `is not None` before comparing a year with 1950?

Keep learning

[Django models](#)

Django Girls Tutorial · tutorial · beginner. Start here: explains what a model is using everyday objects, then builds one step by step.

[Django Model Fields – Common Use Cases and How They Work](#)

freeCodeCamp · article · beginner. Plain-language tour of CharField, IntegerField, DateField and default values, with examples of when to use each.

[Writing your first Django app, part 2](#)

Django docs · tutorial · beginner. The official tutorial step where you create models, add a `__str__` method and a custom method, just like the Book model.

[Models](#)

Django docs · docs · intermediate. The main guide to fields, Meta options and model methods; read it once you have written your first model.

[Model Meta options](#)

Django docs · docs · intermediate. Reference for class Meta settings such as ordering, which controls the default sort order of your table.